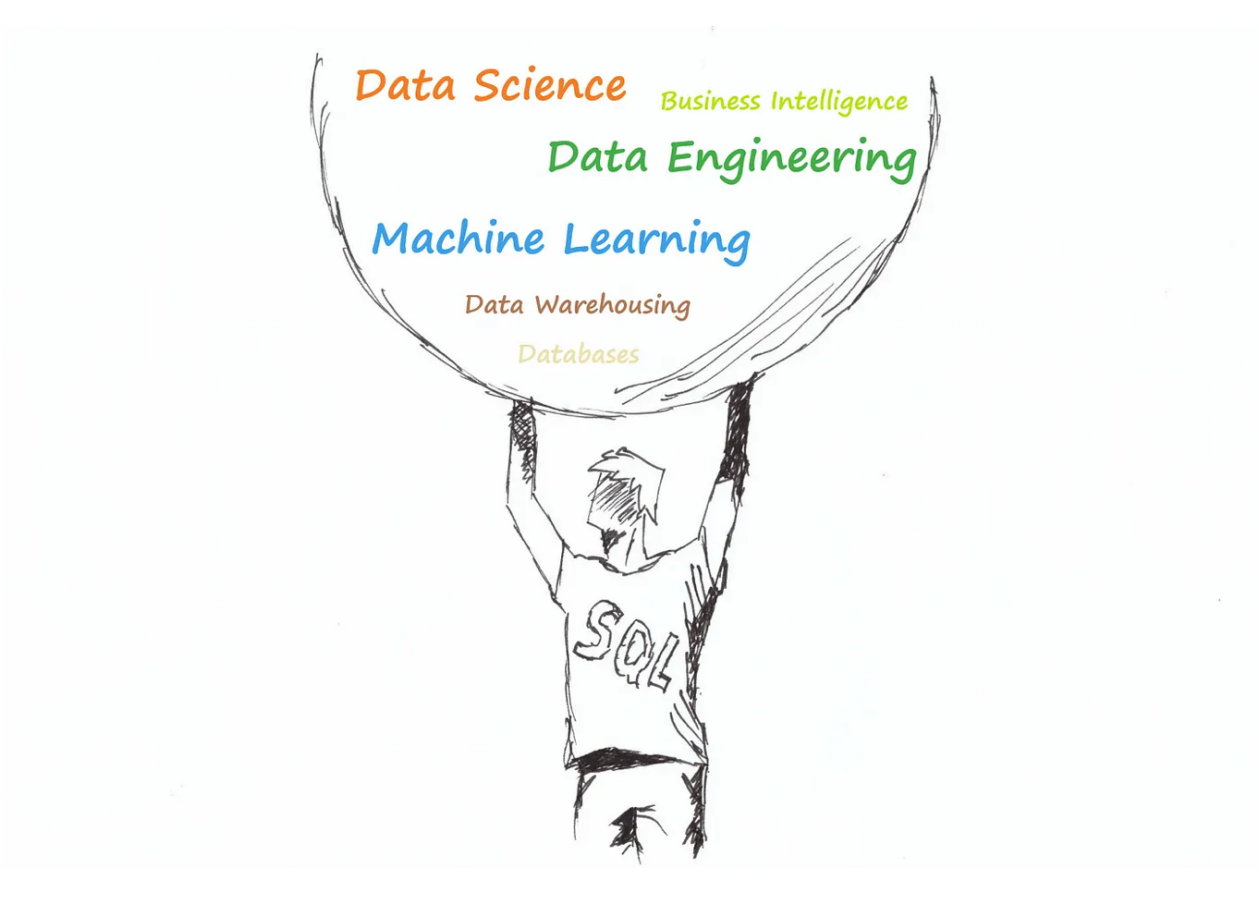
Get unlimited access to the best of Medium for less than $1/week. Become a member



Advanced SQL for Data Professionals

Mariusz Kujawski · Follow
18 min read · Sep 30, 2024

827 12

To start working with data, it is important to learn tools like SQL. Structured Query Language (SQL) is a programming language used for storing, processing, and managing data in relational databases, though its usage has now expanded beyond that, for example, Big Data.

Introduced in the 1970s, SQL remains a core tool in various fields, including data engineering, data analysis, and data science. SQL runs on a wide range of platforms, including SQL Server, Oracle, PostgreSQL, Snowflake, Google BigQuery, and Spark-based platforms such as Databricks.

Even if you plan to become a data engineer or data scientist using only Python, R, or another programming language, it's almost impossible to imagine doing so without knowing SQL.

Common Table Expression

The Common Table Expression (CTE) was introduced into SQL to simplify SQL queries. A CTE is a temporary result set you can reference within a `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement. It allows you to break down complicated queries into smaller, more readable sections.

One of CTE's most useful advanced features is the **recursive CTE**, which enables you to solve problems such as hierarchical data queries (e.g., organizational employee hierarchies) or generating sequential data like calendars.

Below you can find an example of SQL code to create a table and insert records into it:

```
CREATE TABLE Employees
(
  EmployeeID int,
  FirstName string,
  LastName string,
  ManagerID int
)


INSERT INTO Employees VALUES (1, 'Harper', 'Westbrook', NULL);
INSERT INTO Employees VALUES (2, 'Liam', 'Carrington', 1);
INSERT INTO Employees VALUES (3, 'Evelyn', 'Radcliffe', 1);
INSERT INTO Employees VALUES (4, 'Mason', 'Albright', 2);
INSERT INTO Employees VALUES (5, 'Isla', 'Whitman', 2);
INSERT INTO Employees VALUES (6, 'Noah', 'Sterling', 3);
INSERT INTO Employees VALUES (7, 'Ruby', 'Lennox', 3);
INSERT INTO Employees VALUES (8, 'Caleb', 'Winslow', 5);
INSERT INTO Employees VALUES (9, 'Avery', 'Sinclair', 6);
INSERT INTO Employees VALUES (10, 'Oliver', 'Beckett', 6);
```

The code below will generate a result containing information about an employee and his manager. Using a **self-join**, we can match each employee with their corresponding manager based on the `MgrID`.

```
WITH RECURSIVE
  cteReports (EmpID, FirstName, LastName, MgrID, EmpLevel)
  AS
  (
    SELECT EmployeeID, FirstName, LastName, ManagerID, 1
    FROM Employees
    WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmployeeID, e.FirstName, e.LastName, e.ManagerID,
      r.EmpLevel + 1
    FROM Employees e
      INNER JOIN cteReports r
        ON e.ManagerID = r.EmpID
  )
SELECT
  FirstName || ' ' || LastName AS FullName,
  EmpLevel,
  (SELECT FirstName || ' ' || LastName FROM Employees
    WHERE EmployeeID = cteReports.MgrID) AS Manager
FROM cteReports
ORDER BY EmpLevel, MgrID
```

The result will look like this:

Another useful scenario for **CTE** is a **date dimension table** creation, often needed in data warehousing for date-based analysis. A date dimension provides a structured way to handle date ranges, holidays, fiscal years, and other time-related attributes.

The following example demonstrates how to use a **recursive CTE** to generate a date dimension table:

```
with RECURSIVE dates (Date) as (
    Select cast('2024-01-01' as date) as Date, date_part('year', cast('2024-01-0

    union all

    Select date_add(Date, INTERVAL 1 day), date_part('year', date_add(Date, INTE
    from dates
    where Date <= '2025-12-31' -- Put the end date here
)

select *
from dates;
```

In Databricks SQL, recursive CTEs aren't supported. However, you can achieve the same result by using the ***`EXPLODE`*** *function.*

Removing Duplicated Rows

When we work with raw data, it's a common scenario where we need to remove duplicates or retrieve only the latest version. An efficient way to handle this in SQL is to use the `ROW_NUMBER()` window function. This function allows you to assign a unique row number to each row within a partition of your data, based on a specified ordering.

Below is an example of how to use `ROW_NUMBER()` to select only the latest version of a record.

The query will number records, and the newest records will have a value of 1. This way, we can filter only the most recent records from the table.

```
-- sample data
with dane (id, name, age, date) as
(
select 1, 'John Smit', 19, '2020-01-01'
UNION ALL
select 2, 'Eva Nowak', 21, '2021-01-01'
UNION ALL
select 3, 'Danny Clark', 24, '2021-01-01'
UNION ALL
select 4, 'Alicia Kaiser', 25, '2021-01-01'
UNION ALL
select 5, 'John Smit', 19, '2021-01-01'
UNION ALL
select 6, 'Eva Nowak', 21, '2022-01-01'
)


select *
from (
 select
 row_number() over (partition by name order by date) rn,
 *
 from
 dane
 )
where
rn = 1
;
```

Removing Duplicated Rows with a Subquery

There are many ways to achieve the same result in SQL. We can use a **subquery** to identify the most recent record for each group, followed by an **INNER JOIN** to filter out the duplicates.

```
with dane (id, name, age, date) as
(
select 1, 'John Smit', 19, '2020-01-01'
UNION ALL
select 2, 'Eva Nowak', 21, '2021-01-01'
UNION ALL
select 3, 'Danny Clark', 24, '2021-01-01'
UNION ALL
select 4, 'Alicia Kaiser', 25, '2021-01-01'
UNION ALL
select 5, 'John Smit', 19, '2021-01-01'
UNION ALL
select 6, 'Eva Nowak', 21, '2022-01-01'
)


 select
 a.*
 from
 dane a
 inner join (
 select
 name, max(date) last_date from dane group by name

 ) b on a.name = b.name and a.date = b.last_date

;
```

The **sub-query** **`b`** finds the maximum date for each name. Then we use the result of the sub-query to filter and keep only the records with the recent date.

Example:

Finding New Records or Records that Don't Exist

When loading data from a source table, one of the common tasks is to identify and load only new records, i.e., records that don't yet exist in the target table. An effective way to achieve this is to use the **`EXISTS`** command in SQL, which allows you to check if a record from the source already exists in the target table. If it doesn't exist, the record can be inserted or processed.

Test Data for the Exercise

Let's assume that `raw_Employees` is a source table and the `Employees` is a target table.

```
CREATE TABLE Employees
(
  EmployeeID int,
  FirstName string,
  LastName string,
  ManagerID int
);


INSERT INTO Employees VALUES (1, 'Harper', 'Westbrook', NULL);
INSERT INTO Employees VALUES (2, 'Liam', 'Carrington', 1);
INSERT INTO Employees VALUES (3, 'Evelyn', 'Radcliffe', 1);
INSERT INTO Employees VALUES (4, 'Mason', 'Albright', 2);
INSERT INTO Employees VALUES (5, 'Isla', 'Whitman', 2);
INSERT INTO Employees VALUES (6, 'Noah', 'Sterling', 3);
INSERT INTO Employees VALUES (7, 'Ruby', 'Lennox', 3);
INSERT INTO Employees VALUES (8, 'Caleb', 'Winslow', 5);
INSERT INTO Employees VALUES (9, 'Avery', 'Sinclair', 6);
INSERT INTO Employees VALUES (10, 'Oliver', 'Beckett', 6);


CREATE TABLE raw_Employees
(
  EmployeeID int,
  FirstName string,
  LastName string,
  ManagerID int
);
INSERT INTO raw_Employees VALUES (1, 'Harper', 'Westbrook', NULL);
INSERT INTO raw_Employees VALUES (2, 'Liam', 'Carrington', 1);
INSERT INTO raw_Employees VALUES (3, 'Evelyn', 'Radcliffe', 1);
INSERT INTO raw_Employees VALUES (4, 'Mason', 'Albright', 2);
INSERT INTO raw_Employees VALUES (5, 'Isla', 'Whitman', 2);
INSERT INTO raw_Employees VALUES (6, 'Noah', 'Sterling', 3);
INSERT INTO raw_Employees VALUES (7, 'Ruby', 'Lennox', 3);
INSERT INTO raw_Employees VALUES (8, 'Caleb', 'Winslow', 5);
INSERT INTO raw_Employees VALUES (9, 'Avery', 'Sinclair', 6);
INSERT INTO raw_Employees VALUES (10, 'Oliver', 'Beckett', 6);
INSERT INTO raw_Employees VALUES (11, 'Avery', 'Sinclair', 6);
INSERT INTO raw_Employees VALUES (12, 'Oliver', 'Beckett', 6);
```

We can use the `EXISTS` command to filter out records from the source that already exist in the target.

```
SELECT *

FROM
raw_Employees a
WHERE
NOT EXISTS (
  SELECT 1
  FROM
   Employees b
  WHERE
   a.EmployeeID = b.EmployeeID

)
```

This could be useful for reporting purposes or for adding only new records to fact or dimension tables.

An alternative way to achieve the same result is to use a `LEFT JOIN` and a `WHERE` clause. This query will select records that don't exist in the target table.

```
SELECT a.*

FROM
raw_Employees a
left join
 Employees b on a.EmployeeID = b.EmployeeID
WHERE
b.EmployeeID is null;
```

Checking for Existing Values in Other Tables (Active Clients)

When working with data, a common task is validating whether a specific record exists in another table. You might be asked to create a report that shows only active clients. To identify these clients, you can check if a specific `CustomerID` exists in another table.

In such cases, the `EXISTS` command can be used to handle this requirement.

```
-- sample data

CREATE TABLE Customers
(
  CustomerID int,
  FirstName string,
  LastName string
);
INSERT INTO Customers VALUES (1, 'Harper', 'Westbrook');
INSERT INTO Customers VALUES (2, 'Liam', 'Carrington');
INSERT INTO Customers VALUES (3, 'Evelyn', 'Radcliffe');
INSERT INTO Customers VALUES (4, 'Mason', 'Albright');
INSERT INTO Customers VALUES (5, 'Isla', 'Whitman');
INSERT INTO Customers VALUES (6, 'Noah', 'Sterling');
INSERT INTO Customers VALUES (7, 'Ruby', 'Lennox');
INSERT INTO Customers VALUES (8, 'Caleb', 'Winslow');
INSERT INTO Customers VALUES (9, 'Avery', 'Sinclair');
INSERT INTO Customers VALUES (10, 'Oliver', 'Beckett');
INSERT INTO Customers VALUES (11, 'Avery', 'Sinclair');
INSERT INTO Customers VALUES (12, 'Oliver', 'Beckett');

CREATE TABLE orders
(
  OrderID int,
  CustomerID int,
  Quantity int,
  Price decimal(10,2),
  ProductID int
);

INSERT INTO orders VALUES (1,1, 10, 5.2, 1);
INSERT INTO orders VALUES (2,2, 5, 5.2, 1);
INSERT INTO orders VALUES (3,3, 2, 5.2, 1);
INSERT INTO orders VALUES (4,4, 4, 5.2, 1);
INSERT INTO orders VALUES (5,5, 11, 5.2, 1);
INSERT INTO orders VALUES (6,6, 1, 5.2, 1);
INSERT INTO orders VALUES (7,7, 1, 5.2, 1);


-- active clients query

select *
from
  Customers a
WHERE
EXISTS (
  SELECT 1 from orders b
  WHERE
  a.CustomerID = b.CustomerID
)
```

The result should look like this:

Active Clients

If we add `NOT` before the `EXISTS` command, we can retrieve information about inactive clients — those who haven't placed an order.

```
select *
from
Customers a
WHERE
NOT EXISTS (
SELECT 1 from orders b
WHERE
a.CustomerID = b.CustomerID
)
```

Not Active Clients

Filling Gaps in Data Using SQL

One common issue you may encounter is missing entries in data. For instance, when importing data about currencies, gaps may occur because the data is published only on working days. These gaps need to be filled to ensure our calculations and reports function correctly. As shown on the screen below, there are gaps in the data between the records marked in yellow.

Gaps

How Can We Solve It?

To address this issue, we need to use a calendar dimension that includes all required dates. I use a recursive Common Table Expression (CTE) to generate a calendar table.

```
with RECURSIVE calendar (Date) as (
    Select cast('2006-01-01' as date) as Date

    union all

    Select date_add(Date, INTERVAL 1 day)
    from calendar
    where Date <= '2006-01-31' -- Put the end date here
)
, currency (date, price, currency) as (
select cast('2006-01-02' as date) ,3.2582, 'USD'
UNION select cast('2006-01-03' as date) ,3.2488  , 'USD'
UNION select cast('2006-01-04' as date) ,3.1858  , 'USD'
UNION select cast('2006-01-05' as date) ,3.1416  , 'USD'
UNION select cast('2006-01-06' as date) ,3.1507  , 'USD'
--missing data
UNION select cast('2006-01-09' as date) ,3.1228  , 'USD'
UNION select cast('2006-01-10' as date) ,3.128   , 'USD'
UNION select cast('2006-01-11' as date) ,3.1353  , 'USD'
UNION select cast('2006-01-12' as date) ,3.1229  , 'USD'
UNION select cast('2006-01-13' as date) ,3.1542  , 'USD'
--missing data
UNION select cast('2006-01-16' as date) ,3.1321  , 'USD'
UNION select cast('2006-01-17' as date) ,3.1521  , 'USD'
UNION select cast('2006-01-18' as date) ,3.1887  , 'USD'
UNION select cast('2006-01-19' as date) ,3.1772  , 'USD'
UNION select cast('2006-01-20' as date) ,3.1868  , 'USD'
--missing data
UNION select cast('2006-01-23' as date) ,3.1397  , 'USD'
UNION select cast('2006-01-24' as date) ,3.1333  , 'USD'
UNION select cast('2006-01-25' as date) ,3.095   , 'USD'
UNION select cast('2006-01-26' as date) ,3.1253  , 'USD'
UNION select cast('2006-01-27' as date) ,3.1379  , 'USD'
--missing data
UNION select cast('2006-01-30' as date) ,3.1559  , 'USD'
UNION select cast('2006-01-31' as date) ,3.163   , 'USD'
)



 select
  date,
  price,
  currency,
 from
 (
 select
   row_number() over (partition by currency, a.date order by b.date desc) lp,
   a.date,
   b.date org_date,
   b.price,
   currency,
 from
   calendar a left join
   currency b on b.date between a.date - INTERVAL 3 DAY and a.date

 )
 WHERE
 lp = 1
 order by date;
```

Using this query we can fill gaps using data from the previous day. Below you can find the result of the query.

Filled gaps

Finding Employees with the Highest Salary

Finding entities such as employees with the highest salary or customers with the highest spending can be achieved using the `LIMIT` and `ORDER BY` commands. However, what if multiple entities have the same value? With this combination, we may not see all of them in the results. To address this issue, we can use a subquery, as shown in the example below.

```
with Employees  (EmployeeID, Name, Salary) as (
select 1, 'John', 5000
UNION ALL select  2, 'Jane', 7000,
UNION ALL select  3, 'Bob', 4500,
UNION ALL select  4, 'Alice', 9000,
UNION ALL select  5, 'Mike', 9000,
UNION ALL select  6, 'Sara', 8000,
UNION ALL select  7, 'Tom', 6000,
UNION ALL select  8, 'Lucy', 5500
UNION ALL select  9, 'Mary', 5820
UNION ALL select  10, 'Tom', 7890
)

SELECT
    a.*
FROM Employees a
INNER JOIN (
select distinct Salary from Employees order by Salary desc limit 3
) as b
on a.Salary = b.Salary;
```

As you can notice, we aimed to retrieve three employees with the highest salaries, but since two of them have the same salary, they should both be included in the result.

Using the MAX Function with a Subquery

To retrieve the employees who have the highest salary, you can use the `MAX` function to find the maximum salary, and then use a subquery to filter the employees who earn that salary.

```
with Employees  (EmployeeID, Name, Salary) as (
select 1, 'John', 5000
UNION ALL select  2, 'Jane', 7000,
UNION ALL select  3, 'Bob', 4500,
UNION ALL select  4, 'Alice', 9000,
UNION ALL select  5, 'Mike', 9000,
UNION ALL select  6, 'Sara', 8000,
UNION ALL select  7, 'Tom', 6000,
UNION ALL select  8, 'Lucy', 5500
UNION ALL select  9, 'Mary', 5820
UNION ALL select  10, 'Tom', 7890
)

SELECT
    a.*
FROM
Employees a
INNER JOIN (
select max(Salary) salary from Employees
) as b
on a.Salary = b.Salary;

-- OR

SELECT
    a.*
FROM
Employees a
WHERE
Salary = (select max(Salary) salary from Employees)
```

Result:

UNPIVOT

This example comes from Databricks SQL but it will also work with other SQL engines. When you receive data from a source in the format presented below, it will be useful for data modeling purposes to move values from columns to rows.

The example query:

```
with data (productID, I2024, II2024, III2024, IV2024)
AS
(
select 1,100,123,234,4323
UNION ALL
select 2,123,445,33,2212
UNION ALL
select 3,1222,1223,1232,43232
UNION ALL
select 4,111,223,234,213
UNION ALL
select 5,22332,2323,2334,4342
)

SELECT *
FROM
data UNPIVOT INCLUDE NULLS
    (
        sales FOR quarter IN (I2024, II2024, III2024, IV2024)
    );
```

The result will be:

If your database doesn't support `UNPIVOT`, you can achieve the same result using the UNION command.

```
SELECT productID, I2024 as value, 'I2024' as quater
FROM
data
UNION ALL
SELECT productID, II2024 as value, 'II2024' as quater
FROM
data
UNION ALL
SELECT productID, III2024 as value, 'III2024' as quater
FROM
data
UNION ALL
SELECT productID, IV2024 as value, 'IV2024' as quater
FROM
data
```

PIVOT

The operation opposite to `UNPIVOT` is `PIVOT`, which is used to transform data so that row values are presented as columns. This technique is useful for data presentation in reports.

```
with data (year, quarter, region, value) as (
VALUES (2018, 1, 'east', 100),
          (2018, 2, 'east',  20),
          (2018, 3, 'east',  40),
          (2018, 4, 'east',  40),
          (2019, 1, 'east', 120),
          (2019, 2, 'east', 110),
          (2019, 3, 'east',  80),
          (2019, 4, 'east',  60),
          (2018, 1, 'west', 105),
          (2018, 2, 'west',  25),
          (2018, 3, 'west',  45),
          (2018, 4, 'west',  45),
          (2019, 1, 'west', 125),
          (2019, 2, 'west', 115),
          (2019, 3, 'west',  85),
          (2019, 4, 'west',  65)
          )


SELECT year, region, q1, q2, q3, q4
  FROM data
  PIVOT (sum(value) AS sales
    FOR quarter
    IN (1 AS q1, 2 AS q2, 3 AS q3, 4 AS q4))
```

The result:

Pivot Result

For the `PIVOT` operation, we can use the `CASE WHEN` statement in conjunction with the SUM function. This approach is more readable and can be used with SQL engines that do not support the `PIVOT` command.

```
SELECT year, region,
         sum(case when quarter = 1 then value end) AS q1,
         sum(case when quarter = 2 then value end) AS q2,
         sum(case when quarter = 3 then value end) AS q2,
         sum(case when quarter = 4 then value end) AS q4
  FROM data
  GROUP BY year, region;
```

The result:

Pivot result

Compare Row-to-Row: LAG Function

The **`LAG`** function is a window function that allows us to compare the current row with a previous row in a dataset. This is particularly useful for calculations such as Year-over-Year and Month-over-Month comparisons, among other applications.

```
with currency (date, price, currency) as (
select cast('2006-01-02' as date) ,3.2582, 'USD'
UNION select cast('2006-01-03' as date) ,3.2488  , 'USD'
UNION select cast('2006-01-04' as date) ,3.1858  , 'USD'
UNION select cast('2006-01-05' as date) ,3.1416  , 'USD'
UNION select cast('2006-01-06' as date) ,3.1507  , 'USD'
UNION select cast('2006-01-09' as date) ,3.1228  , 'USD'
UNION select cast('2006-01-10' as date) ,3.128   , 'USD'
UNION select cast('2006-01-11' as date) ,3.1353  , 'USD'
UNION select cast('2006-01-12' as date) ,3.1229  , 'USD'
UNION select cast('2006-01-13' as date) ,3.1542  , 'USD'
)

select
  date,
  currency,
  price,
  lag(price) over (order by date) previose_day_price,
  (price - lag(price) over (order by date)) / lag(price) over (order by date) ch

from
currency
```

Expected Result:

The LEAD Function

The **`LEAD`** function works similarly to the **`LAG`** function, but instead of comparing the current record with the previous record, it compares the current record with the next record. This function is particularly useful for analyzing changes over time, such as identifying trends or shifts in data.

```
with currency (date, price, currency) as (
select cast('2006-01-02' as date) ,3.2582, 'USD'
UNION select cast('2006-01-03' as date) ,3.2488  , 'USD'
UNION select cast('2006-01-04' as date) ,3.1858  , 'USD'
UNION select cast('2006-01-05' as date) ,3.1416  , 'USD'
UNION select cast('2006-01-06' as date) ,3.1507  , 'USD'
UNION select cast('2006-01-09' as date) ,3.1228  , 'USD'
UNION select cast('2006-01-10' as date) ,3.128   , 'USD'
UNION select cast('2006-01-11' as date) ,3.1353  , 'USD'
UNION select cast('2006-01-12' as date) ,3.1229  , 'USD'
UNION select cast('2006-01-13' as date) ,3.1542  , 'USD'
)

select
date,
currency,
price,
lead(price) over (order by date) previose_day_price

from
currency
```

Expected Result:

Transforming an Array to Rows

In scenarios where data is stored as an array within a single cell, or when specific transformations are needed, a function like **`EXPLODE`** can be extremely useful for converting array elements into separate rows.

Using EXPLODE in Databricks SQL

Since Databricks does not support recursive Common Table Expressions (CTEs), you can create a calendar table by combining the **`SEQUENCE`** function, which generates an array of values, with the **`EXPLODE`** function to transform that array into rows.

```
select
date,
date_part('YEAR', date) year,
date_part('month', date) month
from (
SELECT
explode(sequence(DATE'2024-01-01', DATE'2024-12-31',INTERVAL 1 DAY)) as  date
) as calendar
```

The result:

UNNEST: DuckDB and BigQuery

In databases like DuckDB and BigQuery, the **`UNNEST`** function is used to flatten arrays or nested structures into separate rows, similar to the **`EXPLODE`** function found in other SQL engines. This transformation is particularly useful when dealing with complex data types, such as arrays or nested JSON structures.

```
SELECT unnest([1, 2, 3]) as qt
```

The result:

NTILE

The **`NTILE`** function is a window function that divides the rows of a result set into a specified number of groups or buckets, ensuring that each group contains as close to an equal number of rows as possible. This function is useful for various analytical tasks, such as dividing data into quartiles or obtaining specific percentiles of records.

```
with Employees  (EmployeeID, Name, Salary) as (
select 1, 'John', 5000
UNION ALL select  2, 'Jane', 7000,
UNION ALL select  3, 'Bob', 4500,
UNION ALL select  4, 'Alice', 9000,
UNION ALL select  5, 'Mike', 12000,
UNION ALL select  6, 'Sara', 8000,
UNION ALL select  7, 'Tom', 6000,
UNION ALL select  8, 'Lucy', 5500
UNION ALL select  9, 'Mary', 5820
UNION ALL select  10, 'Tom', 7890
)

SELECT
    EmployeeID,
    Name,
    Salary,
    NTILE(10) OVER (ORDER BY Salary) AS SalaryQuartile
FROM Employees
;
```

Expected result:

MERGE INTO

The **`MERGE INTO`** command allows you to perform a combination of updates, insertions, and deletions in a target table based on a source table. This command is useful for synchronizing two tables, ensuring that the target table reflects changes made in the source table.

```
use catalog demo;
CREATE TABLE silver.Employees
(
  EmployeeID int,
  FirstName string,
  LastName string,
  ManagerID int
);

CREATE TABLE silver.raw_Employees
(
  EmployeeID int,
  FirstName string,
  LastName string,
  ManagerID int
);

INSERT INTO Employees VALUES (1, 'Harper', 'Westbrook', NULL);
INSERT INTO Employees VALUES (2, 'Liam', 'Carrington', 1);
INSERT INTO Employees VALUES (3, 'Evelyn', 'Radcliffe', 1);
INSERT INTO Employees VALUES (4, 'Mason', 'Albright', 2);
INSERT INTO Employees VALUES (5, 'Isla', 'Whitman', 2);
INSERT INTO Employees VALUES (6, 'Noah', 'Sterling', 3);
INSERT INTO Employees VALUES (7, 'Ruby', 'Lennox', 3);
INSERT INTO Employees VALUES (8, 'Caleb', 'Winslow', 5);
INSERT INTO Employees VALUES (9, 'Avery', 'Sinclair', 6);
INSERT INTO Employees VALUES (10, 'Oliver', 'Beckett', 6);

INSERT INTO raw_Employees VALUES (1, 'Harper', 'Westbrook', NULL);
INSERT INTO raw_Employees VALUES (2, 'Liam', 'Carrington', 1);
INSERT INTO raw_Employees VALUES (3, 'Evelyn', 'Radcliffe', 1);
INSERT INTO raw_Employees VALUES (4, 'Mason', 'Albright', 2);
INSERT INTO raw_Employees VALUES (5, 'Isla', 'Whitman', 2);
INSERT INTO raw_Employees VALUES (6, 'Noah', 'Sterling', 3);
INSERT INTO raw_Employees VALUES (7, 'Ruby', 'Lennox', 3);
INSERT INTO raw_Employees VALUES (8, 'Caleb', 'Winslow', 5);
INSERT INTO raw_Employees VALUES (9, 'Avery', 'Sinclair', 6);
INSERT INTO raw_Employees VALUES (10, 'Oliver', 'Beckett', 6);
INSERT INTO raw_Employees VALUES (11, 'Avery', 'Sinclair', 6);
INSERT INTO raw_Employees VALUES (12, 'Oliver', 'Beckett', 6);
```

To insert all rows from the source table that do not exist in the target table, you can use the following **`MERGE INTO`** statement:

```
MERGE INTO Employees AS target
USING raw_Employees as source
ON target.EmployeeID = source.EmployeeID
WHEN NOT MATCHED THEN INSERT *
```

To both insert new rows and update existing rows in the target table based on the source table, you can use the following **`MERGE INTO`** statement:

```
MERGE INTO Employees AS target USING raw_Employees as source
  ON target.EmployeeID = source.EmployeeID
  WHEN MATCHED THEN UPDATE SET *
  WHEN NOT MATCHED THEN INSERT *
```

Checking if Tables are The Same

When working with data, especially for testing or debugging code, we often need to compare tables. While it can be straightforward for small tables, comparing tables with hundreds of columns can be challenging. In such cases, the **`EXCEPT`** command can be a valuable tool.

```
CREATE TABLE silver.Employees
(
  EmployeeID int,
  FirstName string,
  LastName string,
  ManagerID int
);

CREATE TABLE silver.raw_Employees
(
  EmployeeID int,
  FirstName string,
  LastName string,
  ManagerID int
);

INSERT INTO Employees VALUES (1, 'Harper', 'Westbrook', NULL);
INSERT INTO Employees VALUES (2, 'Liam', 'Carrington', 1);
INSERT INTO Employees VALUES (3, 'Evelyn', 'Radcliffe', 1);
INSERT INTO Employees VALUES (4, 'Mason', 'Albright', 2);
INSERT INTO Employees VALUES (5, 'Isla', 'Whitman', 2);
INSERT INTO Employees VALUES (6, 'Noah', 'Sterling', 3);
INSERT INTO Employees VALUES (7, 'Ruby', 'Lennox', 3);
INSERT INTO Employees VALUES (8, 'Caleb', 'Winslow', 5);
INSERT INTO Employees VALUES (9, 'Avery', 'Sinclair', 6);
INSERT INTO Employees VALUES (10, 'Oliver', 'Beckett', 6);

INSERT INTO raw_Employees VALUES (1, 'Harper', 'Westbrook', NULL);
INSERT INTO raw_Employees VALUES (2, 'Liam', 'Carrington', 1);
INSERT INTO raw_Employees VALUES (3, 'Evelyn', 'Radcliffe', 1);
INSERT INTO raw_Employees VALUES (4, 'Mason', 'Albright', 2);
INSERT INTO raw_Employees VALUES (5, 'Isla', 'Whitman', 2);
INSERT INTO raw_Employees VALUES (6, 'Noah', 'Sterling', 3);
INSERT INTO raw_Employees VALUES (7, 'Ruby', 'Lennox', null);
INSERT INTO raw_Employees VALUES (8, 'Caleb', 'Winslow', 5);
INSERT INTO raw_Employees VALUES (9, 'Avery', 'Sinclair', 6);
INSERT INTO raw_Employees VALUES (10, 'Oliver', 'Beckett', 6);
```

Testing code:

```sql
select * from silver.Employees
except
select * from silver.raw_Employees
```

The result of this command will show records that do not match between the two tables. For instance, if the `ManagerID` for the same row in `raw_Employees` is `NULL`, this discrepancy will be presented in a result.

Expected result:

. . .

Thanks For Reading!

If you found this article insightful, I invite you to show your appreciation by liking it on LinkedIn and clicking the “clap” button. Your support is greatly appreciated.

Data · Data Science · Sql · Data Engineering

827 · 12

Written by Mariusz Kujawski

2.6K Followers · 2 Following

Follow

Cloud Data Architect | Data Engineer https://www.linkedin.com/in/mariusz-kujawski-812bb1103/

Responses (12)

What are your thoughts?

Respond

srikanth b
Oct 22, 2024

For UNPIVOT the query you shared not working

Corrected query below:-

SELECT productID, 'I2024' AS quarter, I2024 AS sales FROM sampl

UNION ALL

SELECT productID, 'II2024' AS quarter, II2024 AS sales FROM sampl

UNION ALL

SELECT productID, 'III2024' AS... more

3 Reply

yeshwanth yeshu
Nov 29, 2024

thanks

1 Reply

Tiana Randriantsoa
Nov 24, 2024

Well thought article

1 Reply

See all responses

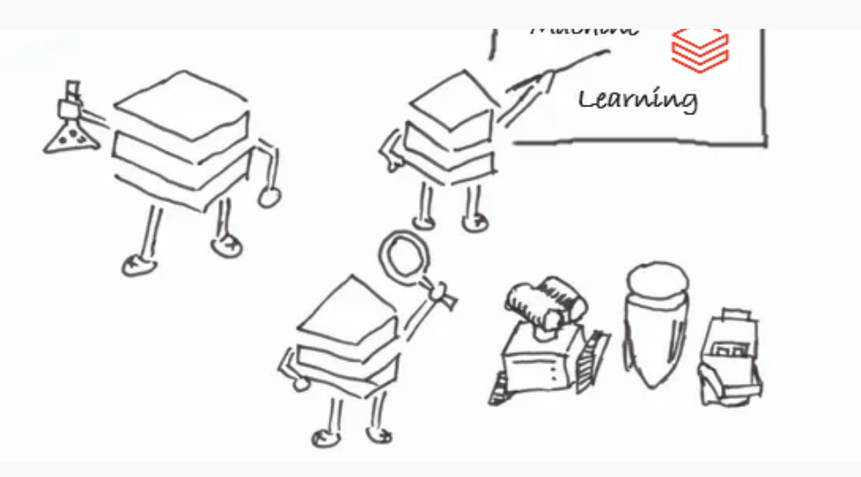
More from Mariusz Kujawski

Mariusz Kujawski

Python in Microsoft Fabric: Data Factory vs Polars and DuckDB

If you’re into data engineering, you probably already know that Microsoft Fabric lets you…

Dec 15, 2024 · 252 · 2

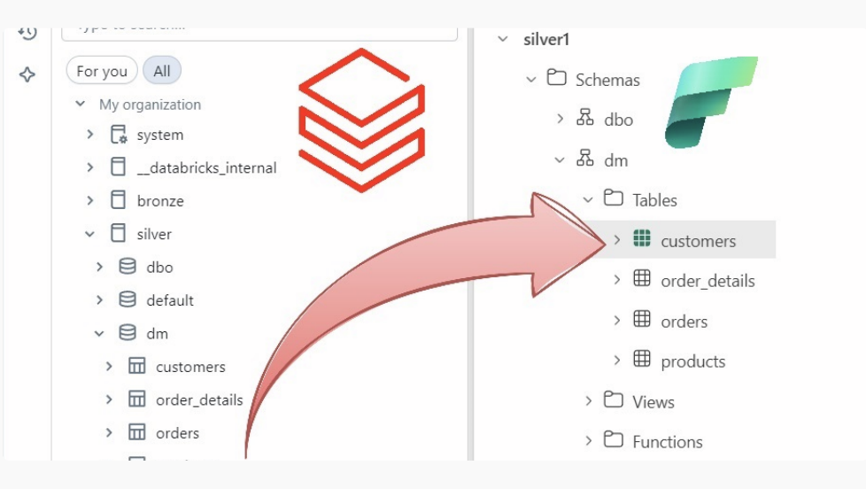


Mariusz Kujawski

Introduction to Databricks: A Beginner’s Guide

In this guide, I’ll walk you through everything you need to know to get started with…

Feb 25, 2024 · 410 · 7

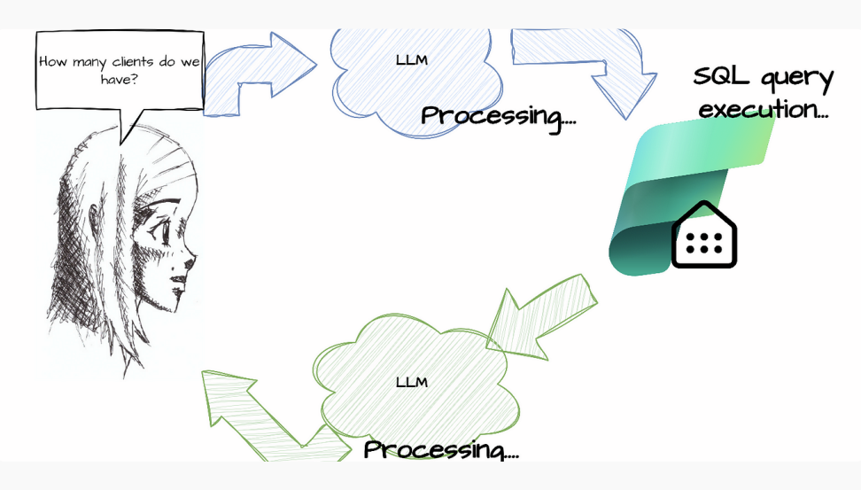


Mariusz Kujawski

Microsoft Fabric and Databricks Mirroring

Are you looking to integrate Databricks with Microsoft Fabric? You’re in luck! Thanks to…

Dec 18, 2024 · 86 · 2


How many clients do we have?
LLM
Processing...
SQL query execution...
LLM
Processing...

Mariusz Kujawski

Building a SQL Bot with LangChain, Azure OpenAI, and Microsoft…

Generative AI is shaking up Business Intelligence (BI) by making data more…

Jan 3 · 15

See all from Mariusz Kujawski

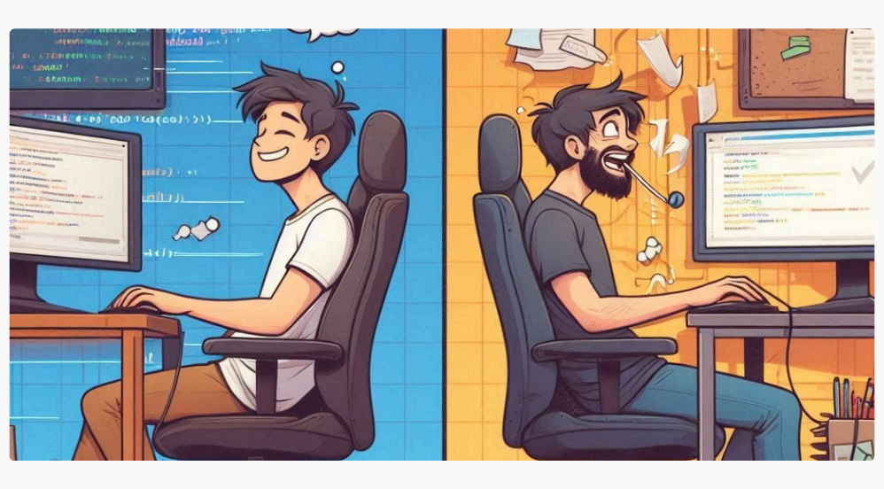
Recommended from Medium

In Python in Plain English by Satyam Sahu

How I Automated Data Cleaning in Python Using Functions and…

Discover the key Python techniques that transformed my data-cleaning workflow fro…

Nov 3, 2024 · 227 · 9



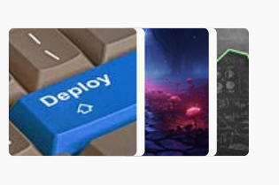
In The Pythoneers by Abhay Parashar

18 Insanely Useful Python Automation Scripts I Use Everyday

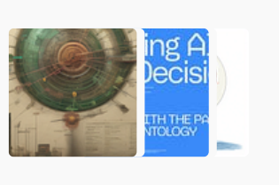
Scripts That Increased My Productivity and Performance Even More

Dec 17, 2024 · 2.8K · 21

Lists



Predictive Modeling w/ Python
20 stories · 1791 saves



data science and AI
40 stories · 320 saves

Practical Guides to Machine Learning
10 stories · 2169 saves

Coding & Development
11 stories · 982 saves

DataScience Nexus

Top Power BI Dashboard Examples

Power BI is one of the most popular business intelligence tools used worldwide for data…

Dec 31, 2024 · 45 · 1

In Level Up Coding by Jacob Bennett

The 5 paid subscriptions I actually use in 2025 as a Staff Software…

Tools I use that are cheaper than Netflix

Jan 7 · 6K · 128

Vijay Gadhave

Must-Have Data Engineering Certifications

Note: If you’re not a medium member, CLICK HERE

Dec 3, 2024 · 371 · 5

In Towards Data Science by Cassie Kozyrkov

Becoming a “real” data analyst

10 differences between amateurs and professional analysts

Apr 14, 2022 · 3.8K · 52

See more recommendations